PROJECT REPORT

TEXT-TO-SPEECH CONVERSION SYSTEM USING PYTHON

Abstract

The Text-to-Speech Conversion System is a Python-based desktop application that converts written text into spoken audio. The application provides a graphical user interface through Tkinter, generates speech using the Google Text-to-Speech (gTTS) library, and plays the resulting audio through an audio playback library such as pygame.

The application includes PLAY, RESET, and EXIT buttons to make it simple to use. It demonstrates the integration of graphical user interface development, text processing, speech synthesis, and audio playback. The system can be used for reading assistance, language learning, and accessibility applications. An internet connection is normally required for speech generation through gTTS.

1. Introduction

Text-to-Speech (TTS) is a technology that converts written text into spoken language. It allows computers to read text aloud, making information easier to access for users who prefer listening to reading.

This project implements a simple TTS application using Python. Users enter text in a graphical window and click the PLAY button to hear the text spoken aloud. The RESET button clears the input, while the EXIT button closes the application.

2. Problem Statement

Reading lengthy text can be time-consuming or difficult for some users. A simple application that converts text into speech can provide an alternative way to consume written information.

The aim of this project is to develop a user-friendly application that accepts text, converts it into audio, and plays the generated speech.

3. Objectives

- Develop a graphical interface for text input.
- Convert written text into spoken audio.

- Generate and save an MP3 audio file.
- Play audio through the computer's speakers or headphones.
- Provide PLAY, RESET, and EXIT controls.
- Validate empty input and handle common errors.
- Demonstrate the practical use of Python libraries.

4. Tools and Technologies

Python: The primary programming language used to implement the application logic.

Tkinter: A standard Python GUI library used to create windows, labels, text boxes, and buttons.

gTTS: A Python library that generates speech audio from text using Google's Text-to-Speech service.

pygame: An audio library used to load and play the generated MP3 file.

Threading: Used in the sample implementation to perform speech generation in the background, helping the GUI remain responsive.

Development environment: Python can be used with VS Code or another Python editor. Running the complete application as a single Python script is generally convenient.

5. System Requirements

Hardware

- Computer or laptop
- Keyboard and display
- Speakers or headphones

Software

- Python
- Tkinter
- gTTS
- pygame
- Internet connection for online speech generation

Install the required libraries using:

```
python -m pip install gTTS pygame
```

Tkinter is normally included with standard Windows Python installations and generally does not require a separate pip installation.

6. System Architecture

The application consists of three logical layers.

1. **Presentation layer:** Displays the window, text box, and buttons using Tkinter.
2. **Application layer:** Validates the entered text and coordinates speech generation and playback.
3. **Speech and audio layer:** Uses gTTS to generate an MP3 file and pygame to play it.

7. Project Workflow

Workflow diagram to insert:

User enters text → Python validates input → gTTS generates speech → Audio is saved as an MP3 file → pygame plays the audio.

The detailed sequence is:

1. The user opens the application.
2. Text is entered into the input area.
3. The PLAY button is clicked.
4. The application checks whether the input is empty.
5. If text is available, gTTS generates speech.
6. The speech is saved as speech.mp3.
7. pygame loads and plays the audio.
8. RESET clears the text and stops playback.
9. EXIT stops playback and closes the application.

8. Working of the Main Functions

Text-to-Speech function: Reads the text, checks for empty input, converts it into speech using gTTS, saves the audio, and plays it.

PLAY button: Calls the Text-to-Speech function when clicked.

RESET button: Stops the current playback and clears the text box.

EXIT button: Stops playback, releases audio resources, and closes the application.

Main window: The Tk() function creates the window. The mainloop() method keeps the application running and responds to user actions.

9. Testing

Test case 1 — Valid input: Enter a sentence and click PLAY. Expected result: speech is generated and played if the online service is available.

Test case 2 — Empty input: Click PLAY without entering text. Expected result: a warning message appears.

Test case 3 — Reset: Enter text and click RESET. Expected result: the text box is cleared and playback stops.

Test case 4 — Exit: Click EXIT. Expected result: the application closes.

Test case 5 — No internet connection: Attempt speech generation while offline. Expected result: the application reports a speech-generation error.

10. Advantages

- Easy-to-use graphical interface.
- Converts written text into speech.
- Supports reading assistance and language learning.
- Demonstrates GUI programming and library integration.
- Can be extended with additional features.

11. Limitations

- An internet connection is normally required by gTTS.
- Speech generation depends on network availability.
- The basic implementation uses a fixed language.
- Very long text may take longer to process.
- More work is needed for advanced audio controls and production-level error handling.

12. Future Enhancements

The application can be improved by adding language selection, pause and resume controls, text-file uploads, progress indicators, audio file naming options, and a more polished interface. Offline speech synthesis could also be explored for use without internet connectivity.

13. Conclusion

The Text-to-Speech Conversion System demonstrates how Python can be used to develop a practical desktop application. Tkinter provides the graphical interface, gTTS generates speech audio, and pygame plays the resulting file. The project provides a foundation for understanding GUI development, function calls, input validation, external libraries, and audio processing.

14. References

1. Python documentation — Tkinter: https://docs.python.org/3/library/tkinter.html
2. gTTS documentation: https://gtts.readthedocs.io/
3. pygame documentation: https://www.pygame.org/docs/
4. Python documentation — Threading: https://docs.python.org/3/library/threading.html